R-107, First Floor, Metro Tower, Vijay Nagar, Indore (M.P.) 452010

Cypher Analytica — Python Full Stack Development Course

Level: Beginner to Job-Oriented Intermediate
Proposed duration: 240 instructor-led hours + approximately 120 hours of independent practice and project work
Suggested schedule: 6 months, approximately 10 instructional hours per week
Prerequisites: Basic computer knowledge; no prior programming experience required.

Core technology stack: Python • Django • Django REST Framework • PostgreSQL • HTML • CSS • JavaScript • React • Git • Docker

Students will learn to build and deploy complete web applications, connecting responsive user interfaces with Python APIs and relational databases.

Module-wise curriculum

Module	Topics covered	Hours
1. Development Fundamentals and Tools	How websites work; frontend, backend, and databases; client–server architecture; HTTP/HTTPS; terminal basics; VS Code; Git and GitHub; repositories, commits, branches, and pull requests	8
2. Python Programming Fundamentals	Variables; data types; operators; input/output; conditions; loops; strings; lists, tuples, sets, and dictionaries; slicing; comprehensions; problem-solving exercises	24
3. Intermediate Python and Code Quality	Functions; arguments; scope; modules; virtual environments; OOP; file handling; exceptions; iterators and generators; decorator basics; logging; debugging; introductory unit testing	20
4. HTML and CSS	Semantic HTML; forms and validation; accessibility fundamentals; selectors; box model; typography; Flexbox; Grid; responsive layouts; media queries; building pages from a design	20
5. JavaScript and Browser Programming	Variables and types; functions; arrays and objects; DOM manipulation; events; form validation; modules; promises; async/await; Fetch API; JSON; error handling; browser storage	24

Module	Topics covered	Hours
6. React Frontend Development	Components; JSX; props; state; event handling; hooks; forms; routing; reusable components; API integration; loading and error states; Context basics; introductory component testing	28
7. SQL and PostgreSQL	Relational database design; keys and constraints; CRUD; joins; grouping; subqueries; normalization; indexes; transactions; parameterized queries; database integration concepts	16
8. Django Backend Development	Project and app structure; URLs; views; templates; models; ORM; migrations; admin; forms; authentication; sessions; static and media files; relationships; query optimization basics	28
9. REST API Development	Django REST Framework; serializers; API views and viewsets; routers; HTTP methods and status codes; validation; authentication; permissions; pagination; filtering; API documentation; Postman	20
10. Full Stack Integration and Security	Connecting React with Django APIs; registration and login; session and token approaches; authorization; protected routes; CORS; CSRF; XSS and SQL injection prevention; secure file uploads; environment variables and secrets	12
11. Testing and Deployment	Backend and API tests; frontend test basics; integration testing; production settings; Docker fundamentals; CI/CD introduction; hosting; HTTPS; database migrations; logs; backup and rollback basics	16
12. Capstone Project and Career Preparation	Requirements; database and API design; implementation; testing; deployment; documentation; GitHub portfolio; project presentation; technical interview practice	24
Total	**Instructor-led sessions and guided practical work**	**240**

Practical projects throughout the course

Stage	Project	Skills demonstrated
Python fundamentals	Expense tracker	Functions, collections, files, validation, and exceptions
HTML and CSS	Responsive personal portfolio	Semantic structure, responsive layouts, and accessibility
JavaScript	Task manager	DOM events, form handling, filtering, and browser storage
React	API-powered dashboard	Components, routing, state, API calls, and error states
Django and PostgreSQL	Student management application	Models, relationships, CRUD, authentication, and admin
REST APIs	Course enrollment API	Serialization, permissions, validation, filtering, and testing

Final capstone: choose one

1. **Learning Management System:** Student and instructor accounts, course listings, enrollment, lessons, assignments, and progress tracking.
2. **Recruitment Portal:** Employer and candidate accounts, job listings, search, applications, and application status.
3. **Inventory and Order Management:** Product catalog, stock tracking, orders, role-based access, and summary reports.
4. **Institute Management System:** Student registration, batches, attendance, fee records, and administrative reports.

Keep the capstone focused on a working minimum viable product. Payment gateways, live chat, and AI features can be optional extensions.

Mandatory capstone deliverables

- Responsive React frontend and Python backend.

- PostgreSQL database with documented relationships.
- Authentication and server-side authorization.
- Validated forms and consistent API error handling.
- Tests covering important workflows and permission checks.
- Deployed application with appropriate production settings.
- GitHub repository with setup instructions, API documentation, and a project demonstration.

Assessment Structure

Component	Weight
Coding assignments and practical exercises	25%
Module projects	25%
Practical assessments	15%
Final capstone project	25%
Technical viva and project presentation	10%
Total	**100%**

Certification Criteria: Minimum 60% overall, completion of mandatory assignments, and successful demonstration of a deployed capstone.

Career preparation

Include code reviews, Git collaboration, debugging exercises, resume and GitHub profile development, mock interviews, and communication practice through project presentations.

Target entry-level roles: Python Developer, Django Developer, Backend Developer, Junior Full Stack Developer, and Web Development Intern. Readiness depends on independent coding ability, project quality, and interview performance.

Optional advanced modules: FastAPI, asynchronous Python, Redis and background tasks, WebSockets, cloud infrastructure, and AI API integration. These should follow the core course rather than crowding the beginner curriculum.